

LabReSiD26 - Hands-On 3

Davide Ferrara - 518629

1 Esercizio 1: Identifica i problemi nel codice di partenza

1.1 1. Cosa succede se l'utente lancia ./sniffer senza argomenti?

La seguente linea converte il nome dell'interfaccia in indice numerico. Lo stesso indice è visualizzabile col comando `ip a`:

```
1  /* Recupera l'indice numerico dell'interfaccia */
2  unsigned int ifindex = if_nametoindex(iface);
```

Nel caso in cui `ifindex` fosse **null** il programma al momento della chiamata di `if_nametoindex(NULL)` andrebbe in Segmentation fault in quanto proverebbe ad accedere un'area di memoria di cui non dispone i permessi.

```
1  sudo ./sniffer
2  [sudo] password for dff:
3  [SNIFFER.c] Sniffing su (null)...
4  Segmentation fault
```

È importante controllare anche il valore di ritorno di `if_nametoindex()`, che ritorna 0 in caso di errore. Se non si controlla, viene passato 0 a `bind()`, che in `AF_PACKET` interpreta l'indice 0 come "tutte le interfacce": il programma non crasha ma sniffa silenziosamente su tutte le interfacce invece di fallire.

```
1  ip a
2  1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue
   state UNKNOWN group default qlen 1000
3     link/loopback 00:00:00:00:00:00 brd
       00:00:00:00:00:00
4  2: enp8s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500
   qdisc fq_codel state DOWN group default qlen 1000
```

```

5      link/ether 74:56:3c:c7:5e:5d brd ff:ff:ff:ff:ff:ff
6 4: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
      qdisc noqueue state UP group default qlen 1000
7      link/ether bc:c7:46:9b:2e:fe brd ff:ff:ff:ff:ff:ff
8      inet 192.168.8.230/24 metric 600 brd 192.168.8.255
      scope global dynamic wlan0

```

Nel mio caso l'indice di wlan0 (la scheda wifi) corrisponde a indice 4.

Senza il controllo su ifindex, come si può notare snifferei su tutte le interfacce:

```

1 sudo ./sniffer foo
2 [SNIFFER.c] Sniffing su foo...
3 [SNIFFER.c] Indice interfaccia 0
4 ^C

```

Con il controllo invece esco dal programma:

```

1 sudo ./sniffer foo
2 [SNIFFER.c] Sniffing su foo...
3 Errore: interfaccia 'foo' non trovata

```

1.2 2. Perché controllare il valore di ritorno di socket() e bind()?

Van controllate per evitare che il programma continui con un file descriptor non valido, causando comportamenti indefiniti alla chiamata di (bind(), recvfrom()).

Esempi di errori di socket() che ho riscontrato sono durante la scrittura del programma sono:

- EPERM: il processo non ha i privilegi necessari (serve sudo o la capability cap_net_raw).

cap_net_raw è una Linux capability che permette di aprire raw socket senza essere root. Invece di eseguire l'intero programma con sudo, si assegna solo questo privilegio specifico al binario:

```

1 sudo setcap cap_net_raw+ep ./sniffer
2 ./sniffer wlan0 # funziona senza sudo

```

Listing 1: Assegnare cap_net_raw al binario

Le capability assegnate al binario sono salvate come extended attribute (security.capability) sull'inode del file e leggibili con getcap:

```

1 getcap ./sniffer
2 ./sniffer cap_net_raw=ep

```

Listing 2: Lettura capability con getcap

Gli errori invece riscontrati con bind():

- ENODEV: l'indice di interfaccia passato non corrisponde a nessun dispositivo.

Il codice qui sotto effettua il controllo di socket() e bind() e su EPERM stampa un messaggio personalizzato.

```

1 int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))
  ;
2 if (sock == -1) {
3     if (errno == EPERM)
4         fprintf(stderr, "Errore: esegui con sudo o
          cap_net_raw\n");
5     else
6         perror("socket");
7     exit(EXIT_FAILURE);
8 }
9
10 if (bind(sock, (struct sockaddr *)&sll, sizeof(sll)) ==
    -1) {
11     perror("bind");
12     close(sock);
13     exit(EXIT_FAILURE);
14 }

```

Listing 3: Controllo socket() e bind()

1.3 3. Perché eth->h_proto stampato direttamente dà un valore sbagliato su x86?

Il campo h_proto arriva dalla rete in **big-endian** (byte più significativo per primo): IPv4 è 0x0800, cioè i byte sul cavo sono [0x08, 0x00].

x86 è **little-endian**: il processore legge il byte all'indirizzo più basso come il meno significativo. Quindi [0x08, 0x00] viene interpretato come 0x0008 = 8, invece del corretto 0x0800 = 2048.

ntohs() (*network to host short*) esegue il byte-swap necessario:

```

1 /* SBAGLIATO: stampa 8 invece di 2048 */
2 printf("  EtherType: 0x%04X\n", eth->h_proto);
3
4 /* CORRETTO: ntohs() swappa i byte e stampa 2048 */
5 printf("  EtherType: 0x%04X\n", ntohs(eth->h_proto));

```

Listing 4: Senza e con conversione

1.4 4. Cosa succede se si omette la bind()?

Dal manuale di `bind(2)`:

When a socket is created with `socket(2)`, it exists in a name space (address family) but has no address assigned to it. `bind()` assigns the address specified by `addr` to the socket referred to by the file descriptor `sockfd`. Traditionally, this operation is called "assigning a name to a socket".

Senza `bind()`, il socket `AF_PACKET` non è associato ad alcuna interfaccia e riceve frame da **tutte le interfacce** attive sul sistema. Con `bind()` il kernel consegna solo i frame dell'interfaccia specificata.

Nella prova pratica la differenza non è evidente perché ricevo frame solo su `wlan0` mentre `enp8s0` è in stato `DOWN` e non genera frame. Su un sistema con più interfacce attive simultaneamente (es. `eth0` e `wlan0`), senza `bind()` si vedrebbero frame da entrambe, mischiati e indistinguibili.

2 Esercizio 2: Aggiungi la gestione degli errori

La gestione degli errori è stata già implementata durante l'analisi dell'Esercizio 1. Il codice risultante è riportato di seguito per completezza.

```

1 /* Controlla argc */
2 if (argc < 2) {
3     fprintf(stderr, "Uso: %s <interfaccia> [arp|ipv4|ipv6\n", argv[0]);
4     exit(EXIT_FAILURE);
5 }
6 const char *iface = argv[1];
7
8 /* Apre il raw socket */

```

```

9  int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))
    ;
10 if (sock == -1) {
11     if (errno == EPERM)
12         fprintf(stderr, "Errore: esegui con sudo o
            cap_net_raw\n");
13     else
14         perror("socket");
15     exit(EXIT_FAILURE);
16 }
17
18 ...
19
20 /* Recupera l'indice dell'interfaccia */
21 unsigned int ifindex = if_nametoindex(iface);
22 if (ifindex == 0) {
23     perror("if_nametoindex");
24     close(sock);
25     exit(EXIT_FAILURE);
26 }
27
28 ...
29
30 if (bind(sock, (struct sockaddr *)&sll, sizeof(sll)) ==
    -1) {
31     perror("bind");
32     close(sock);
33     exit(EXIT_FAILURE);
34 }

```

Listing 5: Gestione errori – sniffer.c

3 Esercizio 3: Decodifica l'EtherType

La funzione `ethertype_name()` riceve il valore `proto` già convertito in host byte order (dopo `ntohs()`) e restituisce la stringa corrispondente al protocollo. Riconosce IPv4, ARP e IPv6; per qualsiasi altro valore restituisce "sconosciuto".

Invece dei magic number si usano le costanti simboliche definite in `<net/ethernet.h>`, già disponibili in host byte order:

```

1  #ifndef ETHERTYPE_IP
2  #define ETHERTYPE_IP    0x0800    /* IP protocol */

```

```

3 #endif
4 #ifndef ETHERTYPE_ARP
5 #define ETHERTYPE_ARP 0x0806 /* Addr. resolution
   protocol */
6 #endif
7 #ifndef ETHERTYPE_IPV6
8 #define ETHERTYPE_IPV6 0x86dd
9 #endif

```

Listing 6: Costanti EtherType da net/ethernet.h (estratto)

Uso semplicemente uno switch case per i vari casi:

```

1 const char *ethertype_name(uint16_t proto) {
2     switch (proto) {
3         case ETHERTYPE_IP:
4             return "IPv4";
5         case ETHERTYPE_ARP:
6             return "ARP";
7         case ETHERTYPE_IPV6:
8             return "IPv6";
9         default:
10            return "sconosciuto";
11    }
12 }

```

Listing 7: ethertype_name() – implementazione con costanti simboliche

Nel loop del programma:

```

1 uint16_t proto = ntohs(eth->h_proto);
2 const char *name = ethertype_name(proto);
3 printf("  EtherType: %d %s\n", proto, name);

```

Listing 8: Uso nel loop di stampa

Esempio di Output:

```

1 Frame #1 (139 byte)
2   DST: BC:C7:46:9B:2E:FE
3   SRC: 94:83:C4:72:F8:D1
4   EtherType: 2048 IPv4
5 Frame #2 (66 byte)
6   DST: 94:83:C4:72:F8:D1
7   SRC: BC:C7:46:9B:2E:FE
8   EtherType: 2048 IPv4

```

Listing 9: Output – Es.3

4 Esercizio 4: Statistiche e interruzione pulita

Alla ricezione di SIGINT (Ctrl+C), l'handler imposta `running = 0` interrompendo il loop. Dopo l'uscita dal loop si chiama `print_stats()` che stampa i contatori per protocollo. Il controllo `if (running == 0)` dentro il loop permette di stampare le statistiche anche se l'interrupt arriva durante l'elaborazione del frame corrente (immagino debba finire l'esecuzione un thread? perché senno mi stampava prima le statistiche e successivamente altre informazioni sniffate, per questo ho fatto così).

```
1 volatile int running = 1;
2 unsigned long cnt_ipv4 = 0;
3 unsigned long cnt_arp = 0;
4 unsigned long cnt_ipv6 = 0;
5 unsigned long cnt_other = 0;
6
7 void print_stats(void) {
8     printf("\nSNIFFING STATS: PROTOCOL | COUNT\n");
9     printf("                IPv4      | %ld\n", cnt_ipv4)
10    ;
11     printf("                ARP        | %ld\n", cnt_arp);
12     printf("                IPv6        | %ld\n", cnt_ipv6)
13    ;
14     printf("                Altri      | %ld\n", cnt_other)
15    ;
16 }
17
18 void handle_sigint(int sig) {
19     (void)sig;
20     running = 0;
21 }
```

Listing 10: Variabili globali, handler e `print_stats()`

I contatori vengono incrementati direttamente dentro `ethertype_name()`, nei rispettivi `case`: la funzione ha quindi un side effect oltre a restituire la stringa. Nel loop basta chiamare `ethertype_name(proto)` e i contatori si aggiornano automaticamente.

```
1 const char *ethertype_name(uint16_t proto) {
2     switch (proto) {
3         case ETHERTYPE_IP:    cnt_ipv4++; return "IPv4";
4         case ETHERTYPE_ARP:   cnt_arp++;  return "ARP";
```

```

5 case ETHERTYPE_IPV6: cnt_ipv6++; return "IPv6";
6 default: cnt_other++; return "sconosciuto";
7 }
8 }

```

Listing 11: ethertype_name() con side effect sui contatori

```

1 while (running) {
2     /* ... recvfrom e parsing ... */
3     uint16_t proto = ntohs(eth->h_proto);
4     const char *name = ethertype_name(proto); /*
5         incrementa contatore */
6     printf(" EtherType: %d %s\n", proto, name);
7
8     if (running == 0)
9         print_stats();
10 }

```

Listing 12: Loop – nessuno switch separato

Esempio di Output:

```

1 sudo ./sniffer wlan0
2 [SNIFFER.c] Sniffing su wlan0...
3 [SNIFFER.c] Indice interfaccia 4
4 Frame #1 (139 byte)
5   DST: BC:C7:46:9B:2E:FE
6   SRC: 94:83:C4:72:F8:D1
7   EtherType: 2048 IPv4
8 Frame #2 (66 byte)
9   DST: 94:83:C4:72:F8:D1
10  SRC: BC:C7:46:9B:2E:FE
11  EtherType: 2048 IPv4
12 ^CFrame #5 (139 byte)
13   DST: BC:C7:46:9B:2E:FE
14   SRC: 94:83:C4:72:F8:D1
15   EtherType: 2048 IPv4
16
17 SNIFFING STATS: PROTOCOL | COUNT
18                      IPv4   | 5
19                      ARP    | 0
20                      IPv6   | 0
21                      Altri   | 0

```

Listing 13: Output – Ctrl+C dopo 5 frame

5 Esercizio 5: Filtraggio da riga di comando

Il secondo argomento è opzionale. `filter` è una variabile `uint16_t` inizializzata a 0 prima del loop: con `filter = 0` il loop non salta nessun frame (nessun filtro attivo). Se `argv[2]` è presente, viene convertito nella costante `EtherType` corrispondente una sola volta prima del loop.

Nel loop basta una riga: se `filter != 0` e il protocollo del frame non corrisponde, il frame viene scartato con `continue`.

```
1 uint16_t filter = 0; /* 0 = nessun filtro */
2 if (argc == 3) {
3     if (strcmp(argv[2], "ipv4") == 0) filter =
        ETHERTYPE_IP;
4     else if (strcmp(argv[2], "arp") == 0) filter =
        ETHERTYPE_ARP;
5     else if (strcmp(argv[2], "ipv6") == 0) filter =
        ETHERTYPE_IPV6;
6     else {
7         fprintf(stderr, "Protocollo non supportato: %s\n",
            argv[2]);
8         close(sock);
9         exit(EXIT_FAILURE);
10    }
11 }
```

Listing 14: – prima del loop]Parsing del filtro da `argv[2]` – prima del loop

```
1 uint16_t proto = ntohs(eth->h_proto);
2 if (filter != 0 && proto != filter) continue;
```

Listing 15: Applicazione filtro nel loop

6 Domande di approfondimento

Q1. Differenza tra `SOCK_RAW` e `SOCK_DGRAM` con `AF_PACKET`

Con `SOCK_RAW` il kernel consegna l'intero frame Ethernet così come viaggia sul cavo, header MAC incluso. È il livello usato dagli sniffer come Wireshark che scende a questo livello tramite `libpcap`.

Con `SOCK_DGRAM` il kernel rimuove l'header Ethernet prima della consegna, fornendo solo il payload. `SOCK_DGRAM` è adatto a messaggi brevi senza connessione (es. server DNS) dove ordine e affidabilità non sono garantiti.

Nel contesto di un sniffer non sarebbe utile, senza header Ethernet non si vedrebbero i MAC address né l'EtherType.

Q2. Comportamento senza bind()

Senza `bind()`, il socket riceve frame da tutte le interfacce con traffico attivo, non solo da quella passata come argomento.

Non sono riuscito ad effettuare la prova pratica poiché l'unica interfaccia collegata alla rete che ho attualmente è la wlan0 mentre eth0 è DOWN come mostrato nell'output del comando "ip a" sopra citato.